Capstone Project: CAP-Based AI Travel Booking Agent

1. Overview

The capstone project challenges trainees to apply everything learned during the 40-hour hands-on workshop by building a production-ready **AI Travel Booking Agent** on the **SAP Cloud Application Programming Model (CAP)**. The agent uses **LangChain** for orchestration, integrates with a mock **S/4HANA Business Partner MCP service**, and exposes itself via the **Agent-to-Agent (A2A) protocol**.

Upon completion, trainees will have built an end-to-end AI agent system that mirrors real-world SAP enterprise architecture — combining CAP services, LLM tool-calling, MCP-based S/4HANA integration, human-in-the-loop approvals, and the A2A interoperability standard.

---

2. Training Context (40 Hours)

The capstone builds on a 40-hour hands-on curriculum covering the full lifecycle of AI agent development on SAP BTP:

Day	Topic	What Was Built
4	First AI Agent	Hello-world agent on CAP with LangChain + SAP AI Core Orchestration
5	Cloud Deployment	MTA build, IAS auth, deployment to SAP BTP Cloud Foundry
6	Tool Calling	LangChain tools wrapping CAP services (read/write via `cds.connect.to()`)
7	A2A Protocol	A2A server with AgentCard, event factories, checkpointer, executor
8	Joule Integration	A2A executor adapted for SAP Joule compatibility
9	MCP Server	Exposing S/4HANA Sales Orders as an MCP endpoint with `@cap-js/mcp`
10	MCP Client	LangChain agent consuming MCP tools via `@langchain/mcp-adapters`
11	Skills & Middleware	Agent skills catalog, progressive disclosure, summarization middleware
12	RAG	Knowledge base ingestion and retrieval via SAP Document Grounding
13	Memory	Short-term (checkpointer) and long-term (store) memory patterns
14	HITL & Refinements	Human-in-the-loop middleware, resilient resume, Joule context cleanup

---

3. Project Description

Build a **CAP-based AI Travel Booking Agent** that:

- **Chats with users** to understand travel intent (origin, destination, dates, preferences)
- **Searches flights** from a local travel database (53 flights across 49 airports, 5 airlines)
- **Retrieves business partner info** from a mock S/4HANA MCP service (728 customers with addresses)
- **Books travel** by calling a transactional CAP action (`createTravelBooking`)
- **Requires human approval** before finalizing any booking (HITL)
- **Exposes itself as an A2A agent** for interoperability with other agent systems

What's Provided

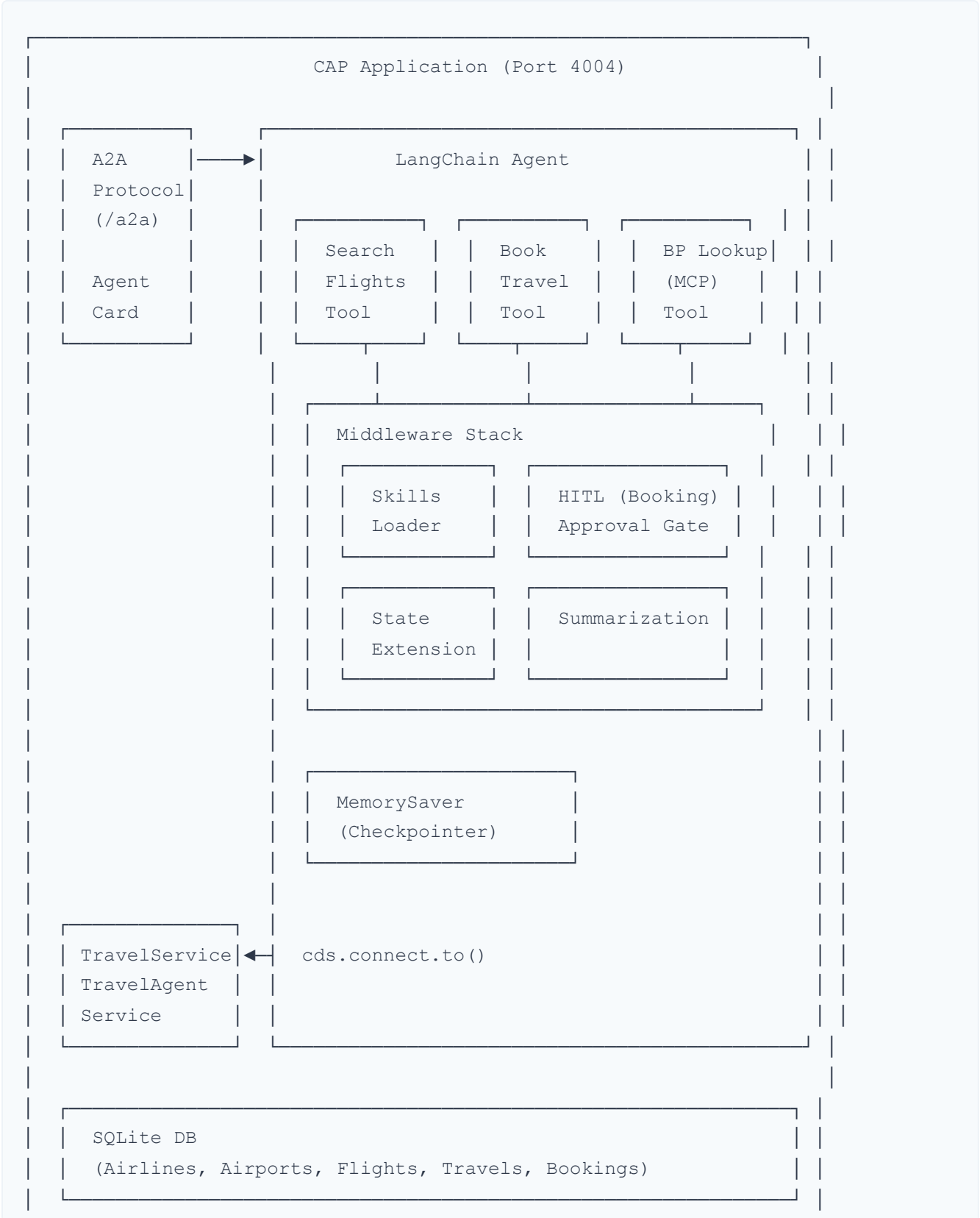
- **cap-travel-agent-starter:** A working CAP backend with:
  - Data model: Airlines, Airports, Flights, Travels, Bookings (with seed data)
  - `TravelService`: OData CRUD for reference data and travels
  - `TravelAgentService.createTravelBooking`: Transactional action to create a travel + bookings atomically
  - SQLite for local development
- **cap-mcp-s4-bupa:** A mock S/4HANA Business Partner MCP server with:
  - `Customers` entity: 728 business partners (name, ID, last modified)
  - `CustomerAddresses` entity: 728 addresses (city, country)
  - MCP endpoint at `http://localhost:4005/mcp/customer`
  - No authentication required (mock mode)

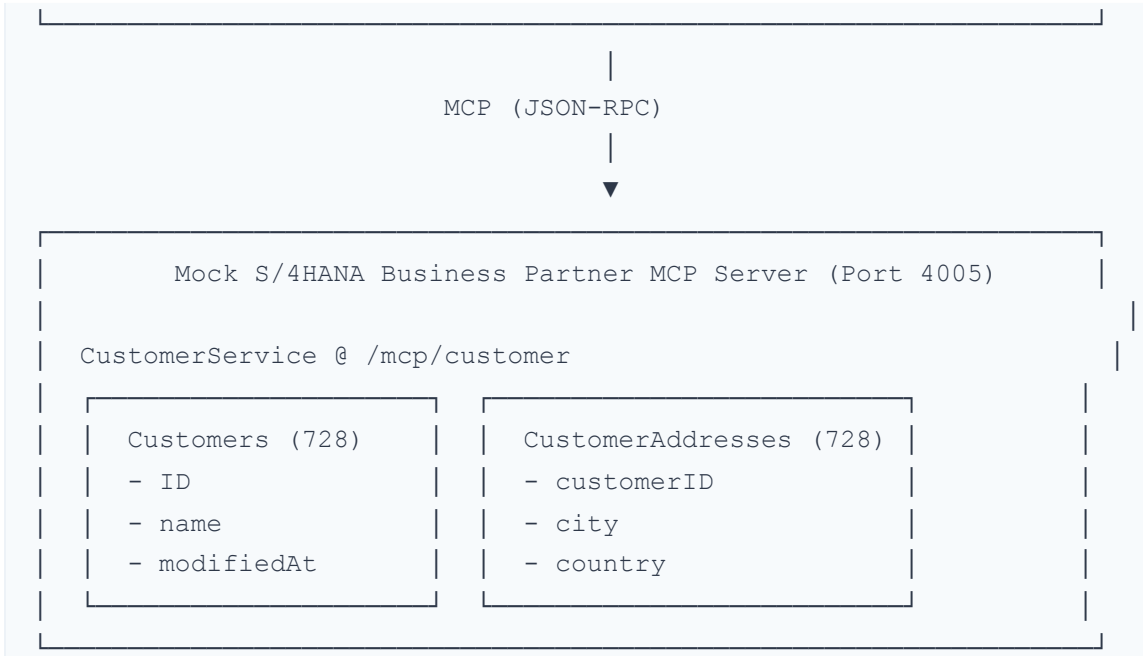
What You Must Build

The starter provides the **backend infrastructure** only. You must implement the **entire agent layer** from scratch, including tool definitions, LLM integration, A2A protocol, and HITL.

---

4. Architecture





Data Flow

1. User sends a message via A2A JSON-RPC endpoint (`/a2a`)
2. Agent executor resolves the conversation context (`thread_id` ← `contextId`) and invokes the LangChain agent
3. The agent determines intent and calls appropriate tools:
   - **Search flights:** Queries `TravelService` via `cds.connect.to()` to find flights matching origin/destination
   - **Lookup business partner:** Calls the MCP client to query the mock S/4HANA service for customer details
   - **Book travel:** Calls `TravelAgentService.createTravelBooking` — but only after HITL approval
4. HITL middleware intercepts booking tool calls and pauses for human confirmation
5. Checkpointer persists conversation state for multi-turn interactions
6. Agent skills provide domain knowledge (how to book, how to retrieve BP) via progressive disclosure
7. Results flow back through A2A status updates and messages

---

5. Implementation Requirements

5.1 LangChain Agent

- Use `@sap-ai-sdk/langchain` (`OrchestrationClient`) to connect to SAP AI Core's Orchestration service
- Implement a system prompt that instructs the agent on its travel agent persona and available capabilities
- Use the ReAct (or similar) agent pattern with tool-calling support

5.2 Tool Definitions

Implement the following tools using LangChain `tool()` definitions with Zod schemas:

Tool	Purpose	Integration Point
`get_airports`	Resolve city names to airport codes	`TravelService.Airports`
`search_flights`	Find flights by origin, destination	`TravelService` via `cds.connect.to()`
`create_travel_booking`	Book a travel + flights in one atomic call	`TravelAgentService.createTravelBooking`
`get_mcp_tools`	Retrieve available MCP tools	Mock S/4 MCP service
`load_skill`	Load domain-specific agent skill on demand	In-app skills catalog

5.3 MCP Client Integration

- Install `@langchain/mcp-adapters` in the agent application
- Build a `MultiServerMCPClient` connecting to the mock S/4 MCP endpoint at `http://localhost:4005/mcp/customer`
- Discover `describe` and `query` as LangChain tools
- Ensure authenticated tool calls by attaching headers via `beforeToolCall` hook where applicable

5.4 A2A Protocol (v0.3)

- Expose an A2A JSON-RPC endpoint at `http://localhost:4004/a2a`

- Publish an **AgentCard** at `/.well-known/agent.json` with:
  - Agent identity (name, description, version)
  - Capabilities and skills
  - `protocolVersion: "0.3.0"`
- Implement event factory helpers for task lifecycle: `submitted`, `working`, `completed`, `failed`, `input-required`
- Build a `LangChainAgentExecutor` that bridges LangGraph state to A2A events:
  - Creates tasks and publishes status updates
  - Maps `contextId` ↔ `thread_id` for checkpointer state
  - Handles `input-required` for HITL interrupts
  - Sends final answer as terminal `status-update` messages
- Mount Agent Card and A2A endpoint via CAP's `bootstrap` event

5.5 Human-in-the-Loop (HITL)

- Register LangChain's `humanInTheLoopMiddleware()` on the `create_travel_booking` tool
- The agent must **pause execution** and request human approval before creating any booking
- The A2A executor must correctly handle `input-required` events and resume on human response
- The booking summary presented to the user should include: flight details, dates, business partner info, and total items

5.6 Agent Skills (Context Management)

- Define a skills catalog with markdown-formatted domain knowledge for:
  - **Travel Booking skill:** How to search flights, interpret results, and book travels
  - **Business Partner skill:** How to retrieve and use customer info from the S/4 MCP service
- Implement a `load_skill` tool the agent calls on demand for progressive disclosure
- Use a skills middleware that appends the skills menu to every model call's system prompt

5.7 Checkpointer (Conversation State)

- Attach LangGraph `MemorySaver` as the checkpointer
- Scope state by `thread_id` (mapped from A2A's `contextId`)
- Enable multi-turn conversations where the agent recalls previous context

---

6. Key Learning Outcomes

Outcome	Description
MCP Integration	Emulates real S/4HANA integration via MCP Gateway — SAP-endorsed architecture that complies with SAP API Policy without directly consuming S/4 OData APIs. The mock service can be replaced with an MCP Gateway for production.
Human-in-the-Loop	Implements approval gates before executing critical business operations (travel booking), demonstrating responsible AI patterns for enterprise workflows.
A2A Protocol	Exposes the agent as an interoperable A2A server discoverable via AgentCard, enabling integration with SAP Joule and other A2A-compatible clients.
Checkpointer	Manages conversation state across multi-turn interactions, enabling context-aware dialogue and interrupt/resume flows.
Tool Calling	Wraps local CAP services as LangChain tools, demonstrating the pattern of bridging domain logic to LLM function-calling.
Agent Skills	Implements progressive disclosure of domain knowledge — keeping the base prompt lean while making detailed instructions available on demand.

---

7. Evaluation Method

Following describes how the trainee's implementation will be evaluated:

1. **Prerequisites:** Clone both repositories, run `npm install` and `npm start` on each
2. **MCP Server:** Start the mock S/4HANA Business Partner MCP server at `http://localhost:4005/mcp/customer`
3. **Agent:** Start the CAP Travel Agent at `http://localhost:4004` with the A2A endpoint at `/a2a`
4. **Automated Test Suite:** A test harness will validate:
   - Successful retrieval of business partner info from the mock S/4 MCP service
   - Correct flight search and booking via the local CAP services
   - A2A protocol compliance (AgentCard discovery, JSON-RPC message format, event lifecycle)
   - HITL flow: agent pauses before booking, resumes on approval, and creates the travel record
5. **Code Review:** Manual assessment of code quality, structure, and adherence to CAP and agent development best practices
6. **Documentation Review:** README file evaluated for clarity and completeness

---

8. Evaluation Criteria

Criterion	Weight	What We Look For
Functionality	**35%**	Agent correctly searches flights, retrieves business partner info, and creates bookings. End-to-end flows work without errors.
A2A Protocol Compliance	**20%**	Proper AgentCard at `/.well-known/agent.json`, A2A JSON-RPC endpoint at `/a2a`, correct event lifecycle (submitted → working → completed/failed/input-required), protocol version 0.3.0.
HITL Implementation	**20%**	Agent pauses before booking and presents a clear summary. Resumes correctly on approval. Uses proper A2A `input-required` state.
Code Quality & Best Practices	**15%**	Well-structured codebase with clear separation of concerns. Agent, tools, skills, and middleware in logical modules. Correct use of CAP patterns (`cds.connect.to()`, service handlers). No hardcoded secrets.
Documentation	**10%**	README includes: project overview, setup instructions, how to start both services, dependencies, assumptions, and usage examples. Clear enough for another developer to run and understand.

9. Bonus Points

The following are optional enhancements that demonstrate deeper understanding:

Bonus	Points	Description
User Preferences Memory	+5%	Agent remembers user preferences across sessions (preferred airlines, seating, meal preferences). Uses LangGraph store for durable, cross-conversation memory.
Data Protection & Privacy	+5%	Implements data masking for sensitive fields (e.g., business partner IDs, personal names) before sending to the LLM. Content filtering to prevent sensitive data leakage.

---

10. Test Prompts

The following prompts will be used during evaluation. Your agent should handle them correctly:

> **Prompt 1:** "Find me a flight from Miami to Havana"

Expected behavior: Resolves MIA and HAV airport codes, searches flights, presents options with airline and timing details. No booking is made.

> **Prompt 2:** "Book me a flight from San Francisco to Frankfurt for March 8th, 2027"

Expected behavior: Resolves SFO and FRA, finds matching flights, retrieves business partner info (or prompts for customer ID), presents booking summary, and **pauses for HITL approval** before creating the booking.

> **Prompt 3:** "I'm going on a family vacation from Venice to Tokyo on August 20th. Book it."

Expected behavior: Resolves VCE and NRT/HND, finds flights, handles the context of "family vacation", requests any missing information (e.g., customer ID, number of passengers), presents booking summary, and pauses for HITL approval.

---

11. Setup & Getting Started

Prerequisites

- **Node.js 18+** and **npm**
- **SAP AI Core service key** (for `@sap-ai-sdk/langchain` Orchestration client)
- **Git**

Step 1: Clone and Start the Mock S/4 MCP Server

```
git clone https://github.com/anselm94/cap-mcp-s4-bupa.git
cd cap-mcp-s4-bupa
npm install
npm start
```

Verify at: `http://localhost:4005/mcp/customer`

Step 2: Clone and Set Up the Starter Template

```
git clone https://github.com/anselm94/cap-travel-agent-starter.git
cd cap-travel-agent-starter
npm install
```

Step 3: Implement the Agent

Build your agent layer within the starter template. Key dependencies you will need:

```
npm install @sap-ai-sdk/langchain @langchain/core langchain @a2a-js/sdk @langchain
```

Step 4: Run the Agent

```
npm start
```

Agent available at:

- A2A endpoint: `http://localhost:4004/a2a`
- Agent Card: `http://localhost:4004/.well-known/agent.json`

Environment Variables

Variable	Required	Description
`AICORE_SERVICE_KEY`	Yes	SAP AI Core service key JSON (base64-encoded or raw)

12. Submission Guidelines

1. **GitHub Repository:** Push your completed project to a **public or private** GitHub repository. Include all source code (except `node_modules/` and secrets).
2. **README.md:** Must include:
   - Project overview and architecture
   - Setup instructions (how to install and run)
   - Dependencies and environment variable requirements
   - Usage examples (sample prompts and expected responses)
   - Assumptions made during development
   - Any limitations or known issues
3. **No Secrets:** Ensure service keys, credentials, and `.env` files are excluded (use `.gitignore`).
4. **Single Repository:** The final submission should be a single repository — the agent code built on top of the starter template.
5. **Submission Link:** Share the GitHub repository URL with the evaluation team.

14. Resources

Resource	Link
Starter Template	https://github.com/anselm94/cap-travel-agent-starter
Mock S/4 MCP Server	https://github.com/anselm94/cap-mcp-s4-bupa
Training Handbooks	Available in the `docs/` directory (Days 4–14)
CAP Documentation	https://cap.cloud.sap/docs/
LangChain JS	https://js.langchain.com/
A2A Protocol	https://a2a-protocol.org/
MCP Specification	https://modelcontextprotocol.io/